Máster:
Inteligencia Artificial

Tercer tetramestre 2026

Materia:
Gestión de proyectos de inteligencia artificial

Actividad 4

Matrícula y nombre del estudiante:
AL07176302 - Kevin Flores Caloch

Fecha: 11/10/2026

Descripción

El estudiante analiza, selecciona e implementa modelos preentrenados de Hugging Face para un caso de uso definido, utilizando Google Colab con GPU como entorno de prototipado rápido. Durante la actividad, se identifican los componentes clave del ecosistema Hugging Face (*Models*, *Datasets* y *Spaces*), se establecen criterios técnicos y prácticos de selección y se implementan al menos tres modelos distintos, los cuales se comparan mediante métricas estándar, como *accuracy*, *precision*, *recall* y *F1-score*.

Asimismo, se evalúa el equilibrio entre precisión y eficiencia, se analiza el desempeño en relación con el contexto del problema y se documentan de forma clara los resultados, las decisiones técnicas y las condiciones de ejecución.
Finalmente, se justifica la selección del modelo más adecuado y se organiza un repositorio en GitHub que garantice la reproducibilidad, validación y mantenimiento del prototipo.

Objetivo

Identificar e implementar el modelo preentrenado más adecuado para un caso de uso específico.

Instrucciones

Selecciona una rama de la inteligencia artificial, por ejemplo, procesamiento de lenguaje natural, visión por computadora, audio o aplicaciones multimodales, como contexto del caso de uso.

1. Cuaderno Colab funcional.
    - Implementa al menos tres modelos con el siguiente contenido:
        - Configuración de GPU.
        - Instalación de dependencias.
        - Autenticación en Hugging Face Hub.
        - Carga de modelos y datasets.
        - Pipeline de inferencia.
        - Cálculo de métricas (accuracy, precision, recall, F1-score).
        - Medición de latencia.
        - Tablas o resúmenes.
        - Conclusiones parciales.

Libro con código adjunto.

2. Documento de análisis comparativo de modelos.
    - Elabora un reporte técnico comparativo con el siguiente contenido:
        - Descripción del caso de uso.
        - Criterios de selección.
        - Tabla comparativa de al menos tres modelos (tarea, tamaño, licencia, precisión, latencia y costo o recursos).
        - Análisis de trade-offs (eficiencia frente a precisión).
        - Decisiones de ajuste mínimo.
        - Justificación del modelo recomendado.
        - Umbrales y riesgos.

Reporte técnico comparativo: análisis de sentimientos en reseñas de películas

1. Descripción del caso de uso
- Problema: clasificar automáticamente reseñas de películas en inglés como positivas o negativas para facilitar el análisis de opiniones de usuarios a gran escala.
- Entrada: texto plano con la reseña de la película.
- Salida: clasificación binaria (positive o negative).

2. Criterios de selección de modelos

Se seleccionaron tres arquitecturas basadas en transformers para evaluar tres frentes fundamentales en el prototipado rápido:
1. Modelo de referencia balanceado (distilbert): un modelo más pequeño que bert pero que retiene gran parte de su capacidad.
2. Modelo de máxima precisión (roberta - siebert): optimizado para capturar semántica compleja en análisis de sentimientos de forma ultra precisa.
3. Modelo de máxima eficiencia (mobilebert): diseñado con estructuras delgadas para ejecutarse con latencias mínimas en cpu/móviles.

4. Tabla comparativa de modelos (resultados reales obtenidos)

Modelo	Tarea	Tamaño	Licencia	Accuracy	Precision	Recall	F1-score	Latencia promedio (ms)	Recursos necesarios
Distilbert	Sentiment analysis	~268 mb	Apache 2.0	90.67%	89.44%	92.90%	91.14%	139.46 ms	Cpu / gpu ligera
Roberta	Sentiment analysis	~1.35 gb	Apache 2.0	91.67%	90.62%	93.55%	92.06%	512.57 ms	Gpu recomendada
Mobilebert	Sentiment analysis	~115 mb	Apache 2.0	89.33%	88.68%	90.97%	89.81%	53.98 ms	Cpu / móvil (ultra-ligero)

3. Análisis de trade-offs (eficiencia vs. Precisión)

- Roberta entrega el mejor f1-score (92.06%), pero su latencia es de 512.57 ms por reseña, lo que representa una carga computacional masiva para sistemas de alta concurrencia.
- Mobilebert reduce la latencia a solo 53.98 ms (casi 10 veces más rápido que roberta) sufriendo una pérdida de rendimiento muy aceptable en f1-score (89.81%).
- Distilbert se ubica en el "punto dulce" con una precisión del 90.67% (muy cercana a roberta) y una latencia cómoda de 139.46 ms.

4. Justificación del modelo recomendado

Se recomienda distilbert para el despliegue de este prototipo en producción. Ofrece una precisión excepcional (superior al 90%) reduciendo la latencia de respuesta de forma considerable comparado con roberta. Su menor consumo de memoria (268 mb frente a 1.35 gb) disminuye drásticamente los costos de infraestructura en la nube.

5. Umbrales y riesgos del prototipo

- Sesgo de dominio: los tres modelos evaluados fueron entrenados en conjuntos de datos provenientes mayormente del inglés formal de hollywood. Textos informales, irónicos o con jergas de redes sociales sufrirán degradaciones en la precisión.
- Truncamiento de texto: las arquitecturas bert limitan la longitud máxima de entrada a 512 tokens. Reseñas excesivamente largas perderán el análisis de las últimas oraciones.
- Métricas mínimas de control: se define un umbral de calidad del 85.0% de f1-score para producción. Si en la monitorización continua el rendimiento cae por debajo de este valor, se deberá iniciar un proceso de ajuste fino (fine-tuning) con datos locales.

3. Dataset y scripts de soporte.
   - Debes incluir:
     - Descripción del origen del dataset.
     - Estructura (descripción de cada campo).
     - Scripts reproducibles (procedimiento mediante el cual se generó o fuente de obtención).

A. Descripción del origen del dataset

El conjunto de datos utilizado para la evaluación de los tres modelos preentrenados corresponde a una muestra controlada de validación de sst-2 (stanford sentiment treebank), el cual forma parte de la suite estandarizada de evaluación de lenguaje natural glue (general language understanding evaluation).

- Fuente oficial: el conjunto se obtiene de manera automatizada a través de los servidores de datos públicos de hugging face en su formato optimizado parquet o en su defecto desde los repositorios de facebook AI research (Meta AI).
- Propósito del dataset: proveer un estándar de referencia confiable para el análisis de sentimientos binario (opiniones positivas frente a negativas) en reseñas cinematográficas.
- Volumen de datos evaluado: se seleccionó de forma determinista un subset de 300 muestras secuenciales de validación para asegurar la estabilidad, robustez estadística de las métricas de evaluación (accuracy, precision, recall y f1-score) y viabilidad de tiempo durante el prototipado rápido.

B. Estructura de campos (sst2_dataset_eval.csv)

El archivo estructurado generado durante la ejecución (sst2_dataset_eval.csv) contiene tres campos fundamentales que soportan la reproducibilidad y el análisis posterior:

1. Sentence (texto / string):
    - Descripción: representa la secuencia de texto original correspondiente a la reseña de la película que se someterá al análisis de sentimientos.
    - Ejemplo: "it 's a charming and often affecting journey ."
2. Label (entero / binario):
    - Descripción: etiqueta de referencia (ground truth) provista por el dataset original para entrenar y validar.
    - Valores: 0 para reseñas negativas y 1 para reseñas positivas.
3. Label_text (texto / categórico):
    - Descripción: traducción de texto amigable de la etiqueta binaria para facilitar la lectura del reporte y análisis analítico.
    - Valores: negative o positive.

C. Scripts reproducibles (procedimiento de obtención)

- Para garantizar la reproducción exacta del dataset sin dependencias de almacenamiento persistente externo, se programó un script automatizado con estrategia de respaldo (fallback):

```python
# 3. Carga del Dataset
import pandas as pd
import urllib.request

try:
    print("Intentando cargar desde el repositorio Parquet de Hugging Face")
    # Cargamos el fragmento de validación del dataset oficial sst2 directamente desde sus splits estáticos de Hugging Face
    parquet_url = "https://huggingface.co/datasets/glue/resolve/main/sst2/validation-00000-of-00001.parquet"
    df = pd.read_parquet(parquet_url)
    # Tomar las primeras 300 muestras para optimizar velocidad y costos
    df = df.head(300).copy()
    df['label_text'] = df['label'].map({0: 'negative', 1: 'positive'})
    print("Dataset cargado con éxito.")
except Exception as e:
    print(f"Fallo la carga desde Parquet ({e}). Aplicando respaldo directo con cabecera de navegador")
    # Descarga directa del subset de validación de SST-2 con User-Agent modificado para evitar el HTTP Error 403
    url = "https://dl.fbaipublicfiles.com/glue/data/SST-2/dev.tsv"
    req = urllib.request.Request(
        url,
        headers={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/58.0.3029.11
    )
    with urllib.request.urlopen(req) as response:
        df_raw = pd.read_csv(response, sep='\t')
    df = df_raw.head(300).copy()
    df.rename(columns={'sentence': 'sentence', 'label': 'label'}, inplace=True)
    df['label_text'] = df['label'].map({0: 'negative', 1: 'positive'})
    print("Dataset cargado con éxito desde el servidor de respaldo de GLUE.")

display(df.head())
```

6. Repositorio en github.
    - Crea un repositorio público o privado que contenga:
        - Readme.md con la descripción del proyecto.
        - Entorno de ejecución.
        - Pasos de ejecución.
        - Conclusiones.
        - Archivos de soporte (requirements.txt, data, notebooks, results y src).